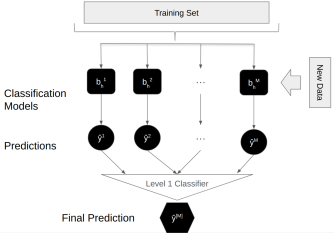
Applied Machine Learning

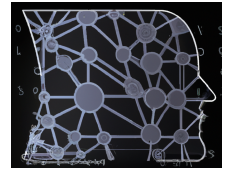
Ensembling Part II



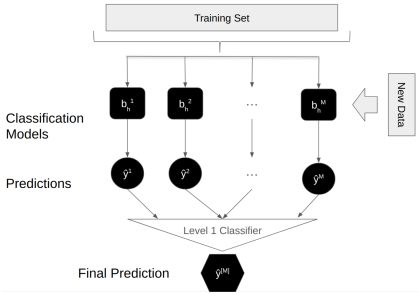
Learning goals

- What is Stacking?
- How can we stack multiple layers?

STACKING



Also called stacked generalization (▸ Wolpert (1992)): train multiple base models (level 0) on the same data and use their predictions as input features for a level 1 model that makes the final ensemble prediction → heterogeneous ensembling

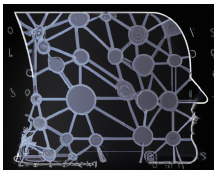


Steps:

1. Base model training (level 0)
2. Collect predictions
3. level 1 model training

In contrast to model averaging, stacking already combines predictions during training.

STACKING



Algorithm Single Layer Stacking

Input : Training data $\mathcal{D} = \left(\left(\mathbf{x}^{(1)}, y^{(1)}\right), \ldots, \left(\mathbf{x}^{(n)}, y^{(n)}\right)\right)$

Output: Stacked Ensemble $\hat{f}^{[M]}$

Step 1: Learn level 0 base models

for $m \leftarrow 1$ **to** M **do**

  Fit base model $b^{[m]}$ on $\mathcal{D}$

end

Step 2: Construct new dataset $\mathcal{D}'$ from $\mathcal{D}$

for $i \leftarrow 1$ **to** n **do**

  Add $(\mathbf{x}'^{(i)}, y^{(i)})$ to new dataset, where $\mathbf{x}'^{(i)} = (b^{[1]}(\mathbf{x}), \ldots, b^{[M]}(\mathbf{x}))$

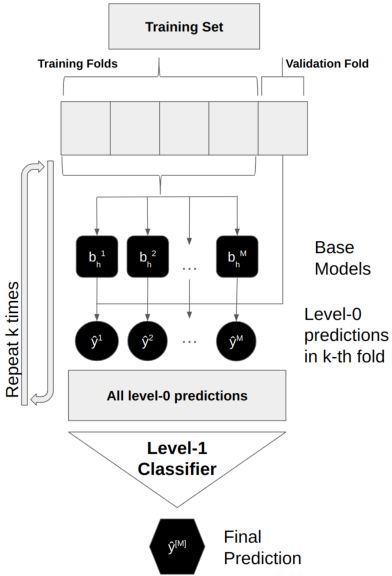
end

Step 3: Fit level 1 model $\hat{f}^{[M]}$ on $\mathcal{D}'$

return $\hat{f}^{[M]}$

NB: We can either drop the original features or keep them when learning the level 1 model.

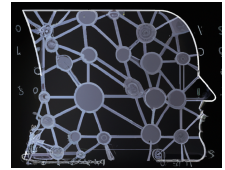
STACKING CROSS-VALIDATED



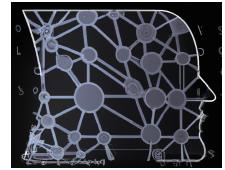
Using cross-validated predictions of the base models

- improve generalization
- reduce overfitting
- enhances base model diversity
- minimizes information leakage

NB: Stacking with a simple hold-out split instead of cross-validation is known as *blending*.



STACKING CROSS-VALIDATED



Algorithm Single Layer Stacking with Cross-Validation

Input : Training data $\mathcal{D} = \left(\left(\mathbf{x}^{(1)}, y^{(1)}\right), \ldots, \left(\mathbf{x}^{(n)}, y^{(n)}\right)\right)$, number of folds K

Output: Stacked Ensemble $\hat{f}^{[M]}$

for $k \leftarrow 1$ **to** K **do**

  Step 1: Learn level 0 base models for $m \leftarrow 1$ **to** M **do**

    Fit base model $b^{[m]}$ on $\mathcal{D}^{(k)}_{\text{train}}$

  end

  Step 2: Add out-of-fold predictions to new dataset $\mathcal{D}'$

  foreach $(\mathbf{x}^{(i)}, y^{(i)}) \in \mathcal{D}^{(k)}_{\text{test}}$ **do**

    Add $(\mathbf{x}'^{(i)}, y^{(i)})$ to new dataset, where $\mathbf{x}'^{(i)} = (b^{[1]}(\mathbf{x}^{(i)}), \ldots, b^{[M]}(\mathbf{x}^{(i)}))$

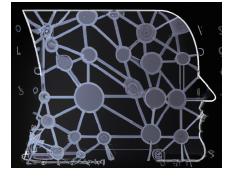
  end

end

Step 3: Fit level 1 model $\hat{f}^{[M]}$ **on** $\mathcal{D}'$

return $\hat{f}^{[M]}$

MULTILEVEL STACKING

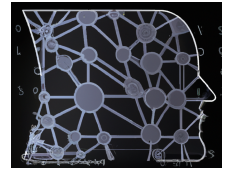


So far: train level 0 base models, use predictions to train the level 1 model.
Now: Also train multiple models at level 1, use predictions to train final level 2 model

1. level 0 base model training
2. Collect predictions of level 0 base models
3. level 1 model training
4. Collect predictions of level 1 models
5. level 2 final model training

NB: In principle we can repeat this training of multiple models at the l-th level indefinitely creating deep stacked ensembles.

HIGHER LEVEL MODELS



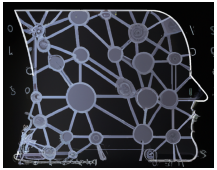
Popular choices of level 1 or level 2 models in stacking include:

- linear / logistic regression
- GES (average the predictions of the models at the previous level)

Higher-level training data:

- Probabilities instead of class labels (see Ting and Witten, 1999)
- Include original dataset **X** or not?

FORWARD PASS / PREDICTION



Algorithm Prediction Using Single Layer Stacking

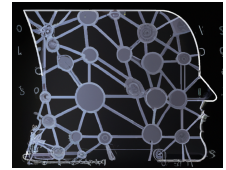
Input : Training data $\mathcal{D} = \left(\left(\mathbf{x}^{(1)}, y^{(1)}\right), \ldots, \left(\mathbf{x}^{(n)}, y^{(n)}\right)\right)$, Test data $\mathbf{X}$
Output: Predictions $\mathbf{y}$
Step 1: Fit level-0 on the whole training data set
for $m \leftarrow 1$ **to** M **do**
  Fit level-0 model $b^{[m]}$ on $\mathcal{D}$
end

Step 2: Make predictions with every level-0 model
for $m \leftarrow 1$ **to** M **do**
  Predict with base model: $\mathbf{y}^m = b_\pi^{[m]}(\mathbf{X})$;
end

Step 3: Construct new dataset $\mathbf{X}'$
for $i \leftarrow 1$ **to** n **do**
  $\mathbf{X}'^i = [\mathbf{x}_1^{(i)}, \ldots, \mathbf{x}_d^{(i)}, \mathbf{x}_1'^{(i)}, \ldots, \mathbf{x}_M'^{(i)}]$
end

Step 4: Make predictions with level 1 model $\hat{f}^{[M]}$ on $\mathbf{X}'$
return $\hat{f}^{[M]}(\mathbf{X})$

TWO QUESTIONS



What other methods do the following level-1 classifiers recover?

- Fit the level-1 training data with a linear model that only depends on one feature, and that feature is chosen such that ρ, our metrics of interest, is optimized.
- The level-1 model is a linear model that assign equal weight $w_m = \frac{1}{M}$ to every input feature.

AUTOGLUON

`https://auto.gluon.ai/`

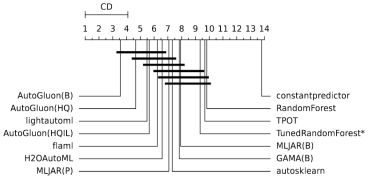
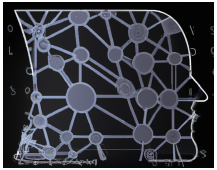


Figure: AutoGluon Performance on AMLB 1h binary classification.

- The de-facto most popular and successful AutoML system
- Developed and maintained by Amazon
- Can also work with multimodal and time-series data
- Does not use HPO to find well performing models
  - Instead, uses a set of default configurations for different learners
- Employs multi-layer stacking using the same set of default models in higher layers as for the base layer
- Uses greedy ensemble selection to combine the predictions of the last layer